CRIMEVISIONAI

CrimeVisionAI — Sample Project Documentation

Comprehensive technical report | Replaces the unrelated CaptionAI example | Prepared 23 October 2026

Project: CrimeVisionAI — Multimodal Forensic Evidence Analysis and Investigation Support System

Document status: Project-specific draft based on the implementation information currently available. Verify placeholders and run-dependent results before formal submission.

Abstract

CrimeVisionAI is a student-built multimodal forensic evidence analysis and investigation-support prototype. It provides a case-centric web workflow for uploading and reviewing audio, image and video evidence. The current implementation combines speech transcription, object detection, image-text scene similarity, image captioning and large-language-model-assisted structuring/Q&A.; It stores case, evidence and analysis metadata in SQLite and keeps uploaded files in local storage. The system aims to reduce first-pass review friction, not to establish facts independently. All generated outputs require human verification. Current limitations include a known investigator-Q&A; grounding issue, lack of speaker diarization, unmeasured performance and unconfirmed report-export/production-hardening features.

1. Introduction

Multimodal evidence review can require a reviewer to inspect different media types with separate tools and manually consolidate observations. CrimeVisionAI explores a unified interface that associates analysis outputs with a case and presents candidate events in chronological context. The project is a prototype for demonstrating applied AI integration and web engineering, rather than a certified forensic platform.

1.1 Problem statement

Reviewers may need to search audio for spoken content, inspect images for objects and scene cues, scan videos for relevant frames, and connect observations across evidence items. These tasks can be repetitive and make it difficult to retrieve information quickly. A single workflow can help organize first-pass analysis, provided that machine-generated results remain traceable and are not confused with verified findings.

1.2 Objectives

• Provide case and evidence organization. • Accept supported audio, image and video files. • Generate timestamped audio transcripts. • Extract candidate visual observations from images and sampled video frames. • Organize candidate events for timeline-oriented review. • Provide investigator Q&A; over available case analysis. • Document limitations, testing and future work transparently.

1.3 Scope

Included in the known baseline are a Flask web application, case/evidence endpoints, local evidence storage, SQLite persistence, audio/image/video analysis and case chat. Formal chain-of-custody certification, speaker diarization, reliable cross-modal identity resolution, autonomous conclusions, production-grade access control, validated forensic accuracy and a confirmed PDF reporting workflow are outside the verified current scope.

2. System overview

The browser UI is served through Jinja2 templates and uses HTML, CSS and vanilla JavaScript. The Flask backend validates requests, coordinates modality-specific analysis and interacts with SQLite and local file storage. Local models handle speech and visual processing; selected interpretation and question-answering tasks use the Groq API. The main flow is: select/create case → upload evidence → analyze → review outputs → inspect timeline → ask a question → verify conclusions against source media.

3. Architecture

Layer	Components	Purpose
Presentation	Browser dashboard, Jinja2 templates, HTML/CSS/JavaScript	Case/evidence interaction, upload, results and Q&A.
Application	Flask 3.x app factory, routes, validation and orchestration	Coordinates request lifecycle and model calls.
Audio processing	Whisper base + timestamped transcription + Groq-assisted structuring	Transcribes audio and creates candidate event descriptions.
Visual processing	RT-DETR, CLIP and BLIP	Object detection, scene-label similarity and captioning.
Video processing	Frame sampling at 1 FPS and visual pipeline reuse	Extracts frame-level observations within configured limits.
Persistence	SQLite tables cases/evidence/analysis_results; local upload directories	Stores metadata, analysis records and media files.
Language integration	Groq API / openai/gpt-oss-20b	Assists with structured interpretation and case Q&A.

4. Technology stack

Layer / Area	Technology / Component	Role / Notes
Frontend	HTML, CSS, JavaScript, Jinja2 templates	Browser-based forensic dashboard; no React/build pipeline reported.
Web backend	Python, Flask 3.x app factory	Routes, request validation, case/evidence workflow and API orchestration.
Database	SQLite via raw sqlite3	Local persistence; tables reported: cases, evidence, analysis_results.
Speech transcription	Whisper base (Hugging Face)	Audio transcription with timestamps; processed in 30-second chunks.
Object detection	RT-DETR (rtdetr-l.pt)	Image/video object detection; configurable confidence presets.
Image-text similarity	CLIP (openai/clip-vit-base-patch32)	Ranks scene/category labels; output is a model estimate, not proof.
Image captioning	BLIP	Generates image/video-frame captions.
LLM integration	Groq API, openai/gpt-oss-20b	Assists with structured interpretation, event/timeline generation and investigator Q&A.
File storage	Local filesystem	Evidence uploads and case-scoped upload paths; upload limit reported as 100 MB.
Runtime	Local CPU with automatic CUDA detection	Hardware availability affects latency; production deployment configuration not yet established.

5. Functional modules

Module	Input	Processing	Output / caveat
Case management	Case details and API requests	Flask case handlers + SQLite	Case records; exact CRUD coverage should be tested.
Evidence intake	Supported media file	Type/size handling, file storage and metadata	Evidence record and local file path.
Audio analysis	WAV/MP3/M4A/FLAC/OGG/ AAC/WMA	Whisper base, 30-second chunks, timestamps, optional LLM structuring	Transcript and candidate events; no diarization.
Image analysis	Image file	RT-DETR confidence presets 0.25/0.4/0.5/0.6; CLIP scene labels; BLIP caption; Groq interpretation	Detections, scene scores, caption and cautious interpretation.
Video analysis	MP4/MOV/AVI/MKV/WEBM	Sample 1 FPS; max 180 seconds / 180 frames; visual models	Frame-level observations and candidate timeline items.
Timeline	Stored analysis/events	Sort/display by available timestamps	Chronological review; cross-source alignment must be validated.
Investigator Q&A	Case question + available analysis	Case chat with Groq integration	Answer may omit relevant entities; known grounding gap.

6. Data model and storage

Three SQLite tables are reported: `cases`, `evidence` and `analysis_results`. The case entity groups related evidence; evidence records identify uploaded media; analysis results associate model output with evidence and/or a case. Uploaded media resides in local directories including `evidence/uploads` and case-scoped upload paths. The database file is reported as `data/crimevision.db`, and a seed case `CASE-2026-MVP-001` has been used. Exact columns, keys, indexes and transaction rules must be taken from the current source schema; this report does not invent them.

7. API and processing behavior

Reported API groups include `/api/audio/analyze`, `/api/image/analyze`, `/api/video/analyze`, case endpoints, evidence upload/analyze/media endpoints and case chat. The precise HTTP methods, JSON schema and remaining route names should be documented from the source code. Flask's reported `MAX_CONTENT_LENGTH` is 100 MB. The development entry point is `run.py`, bound to `127.0.0.1:5000` with debug mode enabled in the reported local configuration.

8. AI model behavior and responsible interpretation

Whisper produces speech text and timestamps, but errors can occur with noise, accents, overlap and low-quality audio. RT-DETR detects classes learned by the model and may miss small, occluded or unusual objects. CLIP returns similarity scores among candidate labels; these are not calibrated probabilities of a real-world event. BLIP captions may omit details or describe ambiguous content incorrectly. Groq-generated interpretations and Q&A; may hallucinate, overgeneralize or omit relevant observations. The UI and reports should distinguish direct model output from generated inference, preserve source links and require human review.

9. Testing strategy and known test evidence

Testing should cover upload validation, case/evidence association, each modality pipeline, timestamp handling, timeline ordering, API failure handling and Q&A; grounding. Previously reported manual video timeline checks validated seeking for example events `evt_v001` and `evt_v015` at approximately 0s and 13s. A known Q&A; issue was observed in which the answer omitted backpack/handbag/person detections around 2.0s while mentioning skis. These observations are limited manual checks, not a comprehensive test suite. No measured latency, throughput, error-rate or memory results are supplied, so performance metrics remain pending.

10. Security, privacy and evidence handling

Use only synthetic, consented or lawfully authorized data during development and demonstration. Keep the application bound to localhost unless properly secured. Never expose Flask debug mode publicly. Keep Groq keys outside source control. Review upload validation, safe filenames, path traversal, size limits, retention and deletion behavior. Add authentication/authorization, HTTPS, audit logging, access controls and secure storage before any multi-user or real-case use. No claim of certified chain of custody or legal admissibility is made.

11. Limitations

• No speaker diarization in the known audio pipeline. • Q&A; grounding/completeness needs correction and regression coverage. • Timeline quality depends on timestamps and model-generated event extraction. • Video is sampled at 1 FPS, which may miss short events between sampled frames. • Processing time depends on CPU/GPU, model initialization and external API latency. • Local SQLite and filesystem storage are prototype-level choices. • PDF report export and 3D visualization are not confirmed complete. • Production authentication, authorization, monitoring and deployment hardening are not verified.

12. Future roadmap

Phase	Enhancement	Exit criteria
Phase 1	Fix Q&A grounding and add evidence-linked answers	Known entity-omission tests pass; unsupported claims are reduced and measurable.
Phase 2	Timeline consistency and multimodal fusion	Source IDs/timestamps retained; duplicate and contradictory events surfaced rather than silently merged.
Phase 3	Structured PDF report export and audit trail	Report export tested; report includes provenance, timestamps, model caveats and review status.
Phase 4	Security and deployment hardening	Debug disabled, secrets managed, authorization, safe upload handling, logging and documented deployment.
Phase 5	Performance and model evaluation	Repeatable benchmark dataset, modality metrics, latency profile and resource measurements.
Phase 6	Advanced capability exploration	Speaker diarization, richer video sampling, identity/track association and 3D visualization only after feasibility review.

13. Installation and run guide

Use the actual repository's dependency file and environment variable names. General procedure: clone the project; create and activate a Python virtual environment; install requirements; configure Groq credentials in the local environment; ensure local models are downloaded/available; initialize database and upload directories; run `python run.py` if this remains the repository's entry point; open `http://127.0.0.1:5000`; test with a small authorized sample. Verify each step on a clean machine and replace this generic sequence with exact source-backed commands before submission.

14. Demonstration plan

Recommended 8–10 minute demo: (1) introduce the problem and safety boundaries; (2) open or create the seed case; (3) upload a synthetic image and inspect detections/caption; (4) run a short audio sample and show timestamped transcript; (5) run a short video and inspect sampled frame timestamps; (6) review the timeline; (7) ask a question and verify the answer against evidence; (8) disclose limitations and roadmap. Pre-download models and prepare fallback screenshots in case external API or hardware latency affects the live demo.

15. Conclusion

CrimeVisionAI demonstrates an integrated prototype for multimodal evidence review using web development, computer vision, speech processing, database persistence and LLM-assisted analysis. Its strongest value is the unified workflow and integration opportunity. Its reliability must be improved through grounded answers, source traceability, systematic evaluation and security hardening before it can be considered for real operational forensic use.

Appendix A — Details requiring final source verification

Insert actual GitHub URL; exact directory tree; Python and dependency versions; complete endpoint methods and schemas; database column/relationship diagram; required environment variable names; clean-machine setup results; named team roles; test screenshots and benchmark outputs; and whether PDF report export is implemented in the latest commit.